

Project Bingo

Overview

Project Bingo is an internal technology modernization initiative intended to improve the way the company stores, searches, and retrieves operational knowledge. The project begins on September 2, 2026, and is expected to run through the end of the calendar year. Bob Doe is the Project Manager and is responsible for coordinating technical implementation, stakeholder communication, testing, and final delivery.

The main objective of Project Bingo is to demonstrate that corporate documents can be converted into structured information suitable for use in an internal knowledge-base system. The project will process documents such as PDF files, Microsoft Word documents, PowerPoint presentations, and HTML pages. Processed content must retain enough metadata for users to identify the original source of retrieved information.

Project Goals

Project Bingo has four primary goals. First, the project must provide a repeatable document preprocessing workflow. A user should be able to place an approved source document into a designated corporate directory and run a documented preprocessing command. The resulting output should be clean, structured, and suitable for downstream retrieval.

Second, the project must preserve document provenance. Each processed chunk should retain useful information about its source, including the original filename and, where available, page or section information. This allows users to trace an answer from the knowledge base back to the original corporate document.

Third, the project must support semantic retrieval. Users should be able to ask questions using language that does not exactly match the wording of the source material. For example, a query asking who is responsible for Project Bingo should retrieve information identifying Bob Doe as the Project Manager.

Finally, the project must operate entirely within the company's self-hosted infrastructure. Corporate documents must not be sent to external AI, embedding, document-processing, or cloud storage services.

Technical Architecture

The preprocessing component uses the open-source Unstructured platform. Raw documents are stored beneath the corporate portion of a shared knowledge vault. Unstructured reads the source document, identifies structural elements such as titles and narrative text, and produces chunked output for later ingestion.

The test environment separates source and processed material into two directories. Original documents are stored beneath `/vault/corporate/raw`, while cleaned output is written beneath

`/vault/corporate/processed`. The preprocessing service receives read-write access only to the corporate subtree and must not access other vault directories.

AnythingLLM provides the downstream knowledge-base layer. It reads processed documents from the corporate directory using a read-only bind mount. AnythingLLM stores its own workspace configuration, vector database, and application state in a separate persistent data directory.

Both services attach to the same pre-existing Docker bridge network. The network name is supplied through an environment variable rather than being hardcoded into either Docker Compose file.

Embeddings and Retrieval

Project Bingo deliberately separates document embeddings from generative language-model inference. AnythingLLM uses a small local embedding model suitable for CPU execution. A MiniLM-class model is sufficient for converting document chunks and user queries into vector representations.

These vectors are stored in a local vector database. When a user submits a question, the question is embedded using the same embedding model. The vector database then searches for document chunks with similar semantic representations.

For example, a user might ask, “Who is responsible for the Bingo project?” The source document does not need to contain that exact sentence. A successful semantic search should still retrieve the section stating that Bob Doe is the Project Manager.

The large generative language model is a separate service. It is used only after relevant document context has been retrieved. This prevents embedding requests from consuming scarce GPU memory and allows the much smaller embedding workload to remain CPU-based.

Security Requirements

Project Bingo follows a least-privilege approach to filesystem access. Neither container should mount the root of the shared knowledge vault. The Unstructured service receives access only to `/vault/corporate`, while AnythingLLM receives the same directory as read-only.

Other vault directories, including shared data, support tickets, learning material, and system skills, are outside the scope of Project Bingo and must remain inaccessible to both containers.

Secrets must also remain outside Docker Compose configuration files. API keys, signing keys, host paths, network names, and environment-specific model configuration should be stored in `.env` files. An `.env.example` file should document all required values without containing real credentials.

Testing and Verification

The initial preprocessing test uses documents containing easily identifiable information about Project Bingo. PDF, DOCX, PPTX, and HTML files should all produce structured output. Tests should confirm that extracted text remains readable and that available metadata such as filename and page number survives preprocessing.

A larger document such as this one is also used to verify chunking behavior. Because the document contains multiple titled sections and several thousand characters of text, a title-aware chunking strategy should produce multiple chunks rather than treating the entire document as a single element.

The configured maximum chunk size should be enforced programmatically. Individual chunks should remain below the documented hard limit while preserving coherent sections whenever possible.

After preprocessing, the resulting content is ingested into the Corporate Test workspace in AnythingLLM. Semantic vector searches should retrieve relevant Project Bingo chunks for queries concerning project management, security, document processing, embeddings, and deployment.

Expected Outcome

Project Bingo is successful when a new operator can deploy both stacks using documented Docker Compose commands, preprocess an approved corporate document, ingest the resulting content into AnythingLLM, and retrieve the correct context through semantic search.

The final system should be reproducible, locally hosted, resource-conscious, and narrowly scoped to the corporate knowledge directory. Bob Doe remains responsible for coordinating the project through final verification and delivery.